

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

EXPERIMENT 8

Student Name: Aryan Yadav

UID: 24BCF10079

Branch: AIT-CSE (FSD)

Section/Group: 24AIT_KRG-G2

Semester: 5

Subject Name: Computer Network

Subject Code: 24CSH-337

Experiment

Implementing a Reliable Connection Using TCP

Aim

To build a TCP client–server application using Python socket programming and observe how a connection-oriented session behaves in practice.

Objective

- Understand the working principles of TCP and its connection-oriented model of communication.
 - Implement a TCP client–server connection using Python's socket module.
 - Establish communication between a server.py and a client.py script over TCP sockets.
 - Observe connection establishment, data exchange, and connection termination during a live TCP session.
-

Theory

TCP (Transmission Control Protocol) is a connection-oriented, reliable transport-layer protocol that guarantees ordered, error-checked delivery of data between two hosts. It follows a client–server model: the server opens a socket, binds it to an IP address and port, and listens for incoming connections, while the client opens its own socket and connects to the server on that same address and port. Once the server accepts the connection, both sides are free to exchange data until either one closes it.

TCP's reliability comes from a handful of underlying mechanisms — sequence numbering, acknowledgements, automatic retransmission of lost segments, and flow control — which together ensure data arrives complete, in order, and without duplication. Before any actual data is sent, TCP runs a three-way handshake to synchronize both ends of the connection:

- SYN — the client sends a synchronization request to the server.
 - SYN-ACK — the server acknowledges the request and sends its own synchronization request back.
 - ACK — the client acknowledges that, and the connection is now established.
-

Practical Steps

1. Open the project folder in a code editor such as VS Code, PyCharm, or IntelliJ.
2. Create two Python files named `server.py` and `client.py`.
3. Paste the server-side code into `server.py` and the client-side code into `client.py`.
4. Save both files.
5. Open two terminals (or a split terminal).
6. In Terminal 1, start the server with `python server.py`.
7. In Terminal 2, start the client with `python client.py`.
8. Exchange a few messages between the client and server CLIs, then type `Over` on the client side to end the session.

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

EXPERIMENT 8

Code

server.py

```
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(("127.0.0.1", 5000)) # bind the server to an IP and port
server_socket.listen(1)

print("Server started")
print("Waiting for a client...")

client_socket, client_address = server_socket.accept()
print("Client connected:", client_address)

while True:
    message = client_socket.recv(1024).decode()
    print("Client:", message)
    if message == "Over":
        break
    reply = input("Enter reply: ")
    client_socket.send(reply.encode())

client_socket.close()
server_socket.close()
print("Connection closed")
```

client.py

```
import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 5000))
print("Connected to server")

while True:
    message = input("Enter message: ")
    client_socket.send(message.encode()) # encode(): string -> bytes
    if message == "Over":
        break
    reply = client_socket.recv(1024).decode()
    print("Server:", reply)

client_socket.close()
print("Connection closed")
```

EXPERIMENT 8

Result

server.py — Terminal 1

```
PS D:\Aryan\CN_Lab\TCP_Experiment> python server.py
Server started
Waiting for a client...
Client connected: ('127.0.0.1', 54831)
Client: Hey, this is Aryan (24BCF10079)
Enter reply: Hi Aryan, handshake completed on this side
Client: Sending a short test message now
Enter reply: Got it, looks like the transfer is clean
Client: Good, I'll wrap up the session
Enter reply: Sounds good, closing up
Client: Over
Connection closed
```

client.py — Terminal 2

```
PS D:\Aryan\CN_Lab\TCP_Experiment> python client.py
Connected to server
Enter message: Hey, this is Aryan (24BCF10079)
Server: Hi Aryan, handshake completed on this side
Enter message: Sending a short test message now
Server: Got it, looks like the transfer is clean
Enter message: Good, I'll wrap up the session
Server: Sounds good, closing up
Enter message: Over
Connection closed
```

Learning Outcome

- Understood the working and characteristics of TCP, including connection establishment, reliable data transfer, and graceful connection termination.
- Built a working TCP client–server application using Python socket programming.
- Analyzed how a connection is established and how data is exchanged between a client and a server over TCP.